Multithreaded Chat Application Using Observer Pattern

Problem:
Real-time communication is difficult with single-threaded systems

Target User
Students / Local network users:

JAYASHRI ROUT

ROLL-09

ENROLL NUMBER-12024052019005

Project Overview

Problem

Handling real-time communication between multiple users is difficult in single-threaded systems.

Solution

We developed a multithreaded chat application using the Observer Pattern to enable real-time communication between multiple clients.

Why It Matters

This system allows efficient communication, supports multiple users simultaneously, and improves scalability.

INTRODUCTION

System Design

Architecture

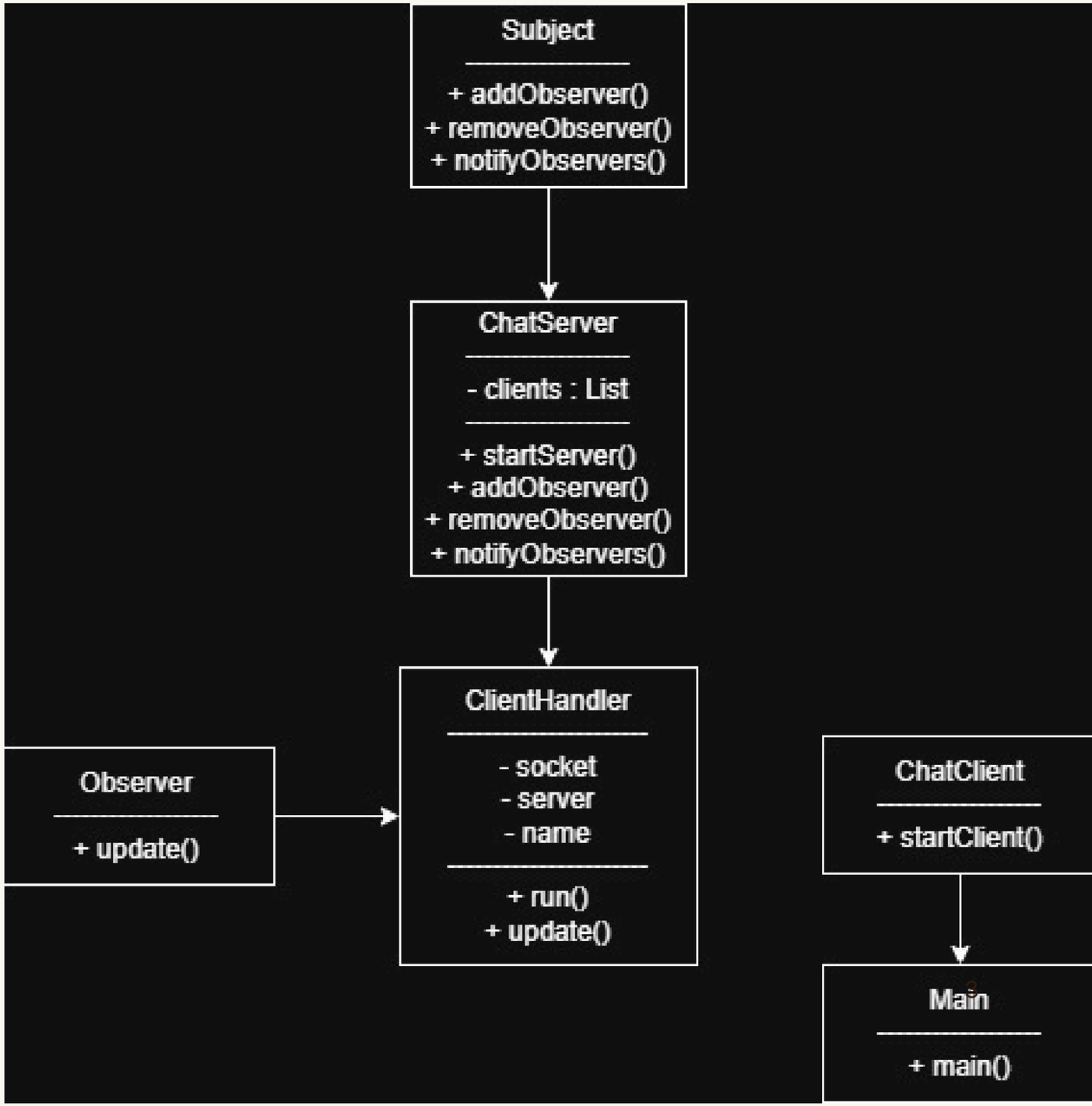
Client → Server → All Clients

Key Components

- ChatServer → manages connections
- ClientHandler → handles each client
- ChatClient → user side
- Observer Pattern → message broadcasting

Structure

All classes organized inside src folder



OOPS Concepts Used

1

Abstraction
Used in: Observer & Subject interfaces
Purpose:Defines common behavior for clients

2

Inheritance
Used in: ClientHandler implements Observer
Purpose: Reuses interface functionality

3

Polymorphism
Used in:update() method
Purpose:Same method works for all clients

Design Pattern

Observer Pattern: Server notifies all clients when a message is received

Technical Highlight

Core Idea

Multithreading + Observer Pattern
Each client runs on a separate thread

Why This design

- Handles Multiple users simultaneously
- ensures real time communication
- Efficient Message Broadcasting
- Scalable and Responsive System

Live Demo

Start Server → Connect Clients → Send Messages → Broadcast and Disconnect

- Server started using terminal
- Multiple clients connected
- Each client enters name

- Client sends message
- Server receives message
- Message broadcast to all clients

- Client disconnect handled
- System continues for other users
- No crash (exception handled)

The End

JAYASHRI ROUT